

BTVN-01: Phân tích hệ thống AI đang dùng - Claude Code

1. Phân loại hệ thống + lập luận theo 3 tiêu chí

Phân loại: Claude Code là một **Agent**.

Đánh giá phân loại dựa trên 3 tiêu chí cốt lõi:

Tiêu chí	Trả lời cho Claude Code	Ví dụ
Ai quyết định bước tiếp theo?	Model	Khi tiếp nhận yêu cầu sửa lỗi mã nguồn, Claude Code tự động lập kế hoạch: đọc tệp tin liên quan, duyệt cấu trúc thư mục, và kích hoạt <code>bash_tool</code> để thực thi bài kiểm thử mà không cần script các bước viết sẵn.
Ai giữ trạng thái?	Model (qua Context) + Hệ thống (lưu tệp tin dự án)	Duy trì ngữ cảnh làm việc qua lịch sử hội thoại và trạng thái files trong không gian làm việc. Tuy nhiên, rủi ro mất ngữ cảnh ban đầu có thể xảy ra khi dung lượng hội thoại vượt giới hạn.
Ai dừng vòng lặp?	Model chốt hoàn thành + Hệ thống kiểm soát ngưỡng giới hạn	Model tự phát tín hiệu dừng khi xác nhận tác vụ hoàn tất dựa trên kết quả đầu ra. Trường hợp lặp vô hạn sẽ bị khống chế bởi lớp giới hạn (timeout, token, số lượt gọi tool) từ hệ thống.

2. Phân tích chi tiết 1 failure mode + tình huống cụ thể

Lỗi chọn phân tích: Thiếu bước xác minh (Verification Failure) → Agent đánh dấu hoàn thành tác vụ khi mã nguồn sinh ra vẫn tồn tại lỗi logic hoặc không vượt qua toàn bộ kiểm thử.

Thực tế diễn biến của case này:

Em nhờ Claude Code sửa giúp hàm tính tổng tiền bị sót mất phần tử cuối. Nó vào sửa code rồi chạy thử test bằng `bash_tool`. Trong bảng log trả về thực ra có 8 bài pass nhưng 2 bài fail. Nhưng vì không có cơ chế bắt nó phải check kỹ log, nó thấy chữ "PASSED" hiện ra nhiều quá liền kết luận luôn: **"Xong rồi nha, test qua hết rồi!"**

Nguyên nhân gây ra lỗi:

Trong tình huống thực tế, khi được yêu cầu sửa lỗi hàm tính toán, Claude Code thực hiện chỉnh sửa mã nguồn và gọi lệnh test thông qua `bash_tool`. Mặc dù kết quả trả về có bài test không vượt qua (failed), Agent nhận diện chuỗi ký tự "PASSED" xuất hiện chủ đạo trong log và kết luận sai lệch rằng toàn bộ script kiểm thử đã đạt yêu cầu, dẫn đến việc thông báo hoàn tất tác vụ.

Người dùng tin tưởng vào phản hồi chốt hoàn thành của AI và triển khai mã nguồn lên môi trường kiểm thử thực tế, làm phát sinh sự cố hệ thống do lỗi logic chưa được khắc phục triệt để.

3. Trình bày 2 lớp harness và cơ chế cụ thể

Lớp 1: **tools**

Mục đích: Đảm bảo mọi tác động đến dự án (đọc, tạo, sửa tệp tin, thi hành lệnh shell) đều được thực hiện thông qua các giao diện có cấu trúc và xác thực chặt chẽ, thay vì chỉ tạo văn bản thuần túy.

Chi tiết kỹ thuật & Cơ chế:

- Hệ thống cung cấp danh mục công cụ có định nghĩa schema rõ ràng (như `bash_tool`, `view`, `str_replace`, `create_file`).
- Model không thể thao tác trực tiếp trên môi trường mà bắt buộc phải yêu cầu gọi tool và phải tuân thủ đúng định dạng dữ liệu.
- Lớp harness tiến hành kiểm tra tính hợp lệ của lệnh trước khi thực thi thực tế và trả kết quả phản hồi về cho Model tiếp tục xử lý.

Lớp 2: **permission**

Mục đích: Kiểm soát hành vi của Agent, phòng ngừa các rủi ro bảo mật, sửa đổi dữ liệu ngoài phạm vi hoặc vô tình thực thi các lệnh nguy hiểm gây tổn hại hệ thống.

Chi tiết kỹ thuật & Cơ chế:

- Hệ thống thiết lập rào cản phân quyền nghiêm ngặt: giới hạn quyền đọc/ghi trong thư mục làm việc được chỉ định, đặt các vùng hệ thống nhạy cảm ở chế độ chỉ đọc (read-only), đồng thời áp dụng cơ chế xác minh (human approval) khi thực thi các lệnh có tác động lớn (như xóa tệp tin hay đọc các file nhạy cảm như `.env`).
- Điều này ngăn chặn triệt để nguy cơ Agent hành động sai lệch gây mất an toàn dữ liệu.